A Multi-Component NLP System for Fake News and Misinformation Detection

Rian Cortez
Holy Angel University
#1 Holy Angel Avenue, Santo Rosario, Angeles City, Pampanga 2009, Philippines
+63 949 581 0362
rian99cortez@gmail.com

Ryence Cortez
Holy Angel University
#1 Holy Angel Avenue, Santo Rosario, Angeles City, Pampanga 2009, Philippines
+63 961 104 6399
cortezryence@gmail.com

Ranz Cuarto
Holy Angel University
#1 Holy Angel Avenue, Santo Rosario, Angeles City, Pampanga 2009, Philippines
+63 947 893 9439
cuartoranz@gmail.com

Mark Darius David
Holy Angel University
#1 Holy Angel Avenue, Santo Rosario, Angeles City, Pampanga 2009, Philippines
+63 969 406 4581
markdariusdavid23@yahoo.com

CCS Concepts

• **Computing methodologies → Natural language processing → Information extraction**

• **Computing methodologies → Machine learning → Neural networks**

• **Computing methodologies → Artificial intelligence → Reinforcement learning**

Keywords

fake news detection; misinformation; natural language processing; text classification; transformer; contextual bandit; reinforcement learning; convolutional neural network; named entity recognition.

1. DATA ACQUISITION AND PREPARATION

The LIAR dataset was successfully downloaded and loaded across all three splits: 10,240 training samples, 1,284 validation samples, and 1,267 test samples. The data pipeline handles TSV loading, label normalization, and text cleaning for both the Text-CNN and DistilBERT branches.

Class distribution is reasonably balanced across all splits, with half-true being the most common label and pants-fire the least frequent at around 8% across splits. Statement lengths average 18 words, with a maximum of 467 words and a minimum of 2 words, the short average length is consistent with the political claim format of the dataset. Republicans are the most represented party (4,497 statements) followed by Democrats (3,336), with Barack Obama being the most frequently mentioned speaker across training data.

Code :

```
# src/data_pipeline.py
import os
import pandas as pd

COLUMNS = [
    "id", "label", "statement", "subject", "speaker",
    "speaker_job", "state", "party", "barely_true_count",
    "false_count", "half_true_count", "mostly_true_count",
    "pants_fire_count", "context"
]

LABEL_MAP = {
    "pants-fire": 0, "false": 1, "barely-true": 2,
    "half-true": 3, "mostly-true": 4, "true": 5
}

DATA_DIR = os.path.join(os.path.dirname(__file__), "..", "data", "raw")

def load_split(split="train"):
    filepath = os.path.join(DATA_DIR, f"{split}.tsv")
    df = pd.read_csv(filepath, sep="\t", header=None, names=COLUMNS)
    df = df.dropna(subset=["statement", "label"])
    df = df[df["label"].isin(LABEL_MAP.keys())]
```

```
    df["label_id"] = df["label"].map(LABEL_MAP)
    df["statement"] = df["statement"].str.strip()
    return df.reset_index(drop=True)

def load_all_splits():
    splits = {}
    for split in ["train", "valid", "test"]:
        splits[split] = load_split(split)
        print(f"[{split}] loaded: {len(splits[split])} samples")
    return splits
```

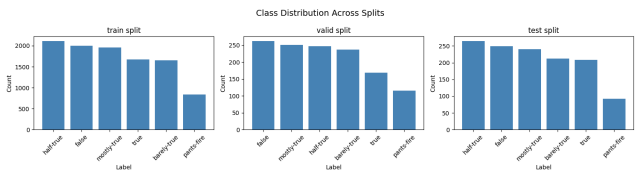

Figure 1: Class distribution across training, validation, and test splits of the LIAR dataset.

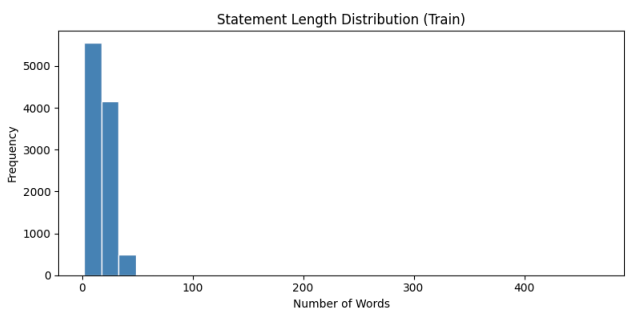

Figure 2: Statement length distribution (in words) across the training split.

2. BASELINE RESULTS

Two baselines were trained and evaluated on the held-out test set.

The non-DL baseline is a Logistic Regression classifier using TF-IDF bigram features with a vocabulary of 10,000 terms. It achieves a test accuracy of 0.2478 and a test Macro-F1 of 0.2214. Per-class performance is weakest on pants-fire (F1: 0.08), which is consistent with that class being the least represented in training.

Code:

```
# Logistic Regression baseline (TF-IDF)

from sklearn.linear_model import LogisticRegression

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics import f1_score, accuracy_score

tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf  = tfidf.transform(X_test)


lr = LogisticRegression(max_iter=1000, random_state=42)

lr.fit(X_train_tfidf, y_train)

y_pred_test = lr.predict(X_test_tfidf)


print(f"Test Accuracy : {accuracy_score(y_test, y_pred_test):.4f}")

print(f"Test Macro-F1 : {f1_score(y_test, y_pred_test, average='macro'):.4f}")
```

Figure 3: Terminal output showing Logistic Regression baseline evaluation results on the held-out test set.

The DL baseline is a Text-CNN following Kim [2], implemented from scratch with three parallel convolutional filter banks over 300-dimensional GloVe embeddings. It achieves a test accuracy of 0.2478 and a test Macro-F1 of 0.2384, outperforming the Logistic Regression baseline on Macro-F1. Training loss decreases steadily across 10 epochs, however validation F1 peaks at epoch 2 (0.2407) and fluctuates afterward, indicating overfitting. Early stopping will be applied in Week 3 to address this.

Code:

```
# src/models/text_cnn.py

import torch

import torch.nn as nn


class TextCNN(nn.Module):

    def __init__(self, vocab_size, embed_dim, num_classes,
                 kernel_sizes=[3,4,5], num_filters=128, dropout=0.5):
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
    self.convs = nn.ModuleList([
            nn.Conv1d(embed_dim, num_filters, k) for k in kernel_sizes
    ])
    self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(num_filters * len(kernel_sizes), num_classes)

  def forward(self, x):
    x = self.embedding(x).permute(0, 2, 1)
        x = [torch.relu(conv(x)).max(dim=2)[0] for conv in self.convs]
    x = torch.cat(x, dim=1)
    x = self.dropout(x)
    return self.fc(x)
```

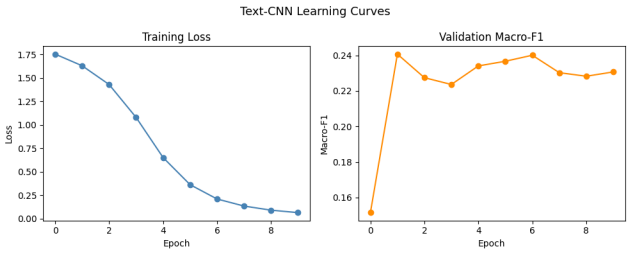

Figure 4: Text-CNN training loss (left) and validation Macro-F1 (right) across 10 epochs.

Table 1: Test accuracy and Macro-F1 results across all models.

Model	Test Accuracy	Test Macro-F1
Logistic Regression (TF-IDF)	0.2478	0.2214
Text-CNN (GloVe 300d)	0.2478	0.2384
DistilBERT (fine-tuned)	TBD	TBD

3. NLP COMPONENT: NAMED ENTITY RECOGNITION

The NER module was scaffolded using spaCy's en_core_web_sm model and run over the full test set. Of the 1,267 test statements, 38.9% contain at least one PERSON entity, 23.8% contain an ORG entity, and 30.9% contain a GPE (location) entity. Barack Obama and Donald Trump are the most frequently mentioned persons. Medicare and Congress are the most common organizations. Texas is the most mentioned location.

A slice analysis was conducted using Logistic Regression predictions to examine whether classification performance varies by entity presence. Statements referencing specific locations (GPE Macro-F1: 0.2011) and organizations (ORG Macro-F1: 0.2068) perform worse than statements without those entities, suggesting the model struggles more with claims tied to specific institutional or geographic contexts.

Code:

```
# src/models/ner_module.py
import spacy
from collections import Counter

nlp = spacy.load("en_core_web_sm")

def extract_entities(text):
    doc = nlp(text)
    entities = {"PERSON": [], "ORG": [], "GPE": [], "OTHER": []}
    for ent in doc.ents:
        if ent.label_ in entities:
            entities[ent.label_].append(ent.text)
        else:
            entities["OTHER"].append(ent.text)
    return entities

def run_ner_pipeline(df):
    df = df.copy()
    df['entities']  = df['statement'].apply(extract_entities)
        df['has_person'] = df['entities'].apply(lambda x: len(x['PERSON']) > 0)
    df['has_org']   = df['entities'].apply(lambda x: len(x['ORG']) > 0)
    df['has_gpe']   = df['entities'].apply(lambda x: len(x['GPE']) > 0)
    return df
```

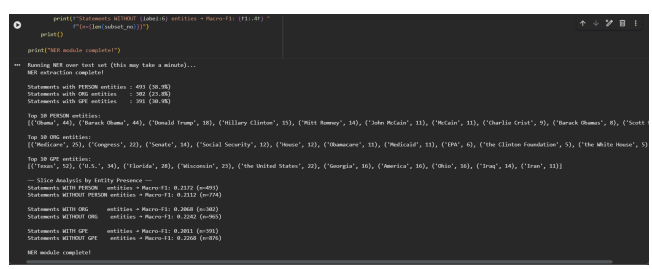

Figure 5: Terminal output showing NER entity frequency counts and slice-level Macro-F1 analysis across entity types.

4. RL COMPONENT: CONTEXTUAL BANDIT AGENT

A contextual bandit agent with epsilon-greedy exploration was implemented and trained over 500 episodes. The agent selects a confidence threshold from a discretized action space of nine values between 0.1 and 0.9. The reward function penalizes false

negatives at -2.0 and false positives at -0.5, reflecting the asymmetry of the detection task.

After training, the agent converged on a threshold of 0.8, selecting it 378 out of 500 times. The bandit agent accumulates higher cumulative reward than a fixed threshold baseline of 0.5 across all episodes. Note that the agent currently runs on simulated softmax outputs sampled from a Dirichlet distribution and will be re-evaluated with real DistilBERT probability outputs in Week 3.

Code:

```
# src/rl_agent.py
import numpy as np


THRESHOLDS = np.round(np.arange(0.1, 1.0, 0.1), 2)


def get_reward(pred_label, true_label, threshold, max_prob):
    if max_prob < threshold:
        return -0.1
    if pred_label == true_label:
        return 1.0
    if true_label in [0, 1] and pred_label not in [0, 1]:
        return -2.0
    return -0.5


class ContextualBandit:
    def __init__(self, n_actions, epsilon=0.1):
        self.n_actions     = n_actions
        self.epsilon       = epsilon
        self.q_values      = np.zeros(n_actions)
        self.action_counts = np.zeros(n_actions)

    def select_action(self):
        if np.random.random() < self.epsilon:
            return np.random.randint(self.n_actions)
        return np.argmax(self.q_values)

    def update(self, action, reward):
        self.action_counts[action] += 1
        self.q_values[action] += (
            (reward - self.q_values[action]) /
            self.action_counts[action]
        )
```

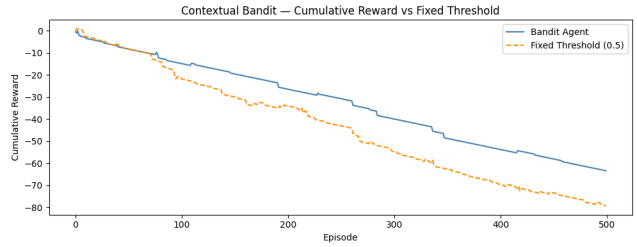

Figure 6: Contextual bandit cumulative reward versus fixed threshold baseline (0.5) across 500 training episodes.

REFERENCES

[1] Kaliyar, R. K., Goswami, A., and Narang, P. 2021. FakeBERT: Fake news detection in social media with a BERT-based deep learning approach. *Multimedia Tools and Applications* 80, 8 (Jan. 2021), 11765–11788. DOI= https://doi.org/10.1007/s11042-020-10183-2

[2] Kim, Y. 2014. Convolutional neural networks for sentence classification. In *Proceedings of EMNLP 2014*. ACL, Doha, Qatar, 1746–1751. DOI= https://doi.org/10.3115/v1/D14-1181

[3] Mao, Y., Wang, Z., Liu, W., Lin, X., and Hu, W. 2021. BanditMTL: Bandit-based multi-task learning for text classification. In *Proceedings of ACL-IJCNLP 2021*. ACL, Online, 5506–5516. DOI= https://doi.org/10.18653/v1/2021.acl-long.428

[4] Shu, K., Sliva, A., Wang, S., Tang, J., and Liu, H. 2017. Fake news detection on social media: A data mining perspective. *ACM SIGKDD Explorations Newsletter* 19, 2 (Sep. 2017), 22–36. DOI= https://doi.org/10.1145/3137597.3137600

[5] Thorne, J., Vlachos, A., Christodoulopoulos, C., and Mittal, A. 2018. FEVER: A large-scale dataset for fact extraction and verification. In *Proceedings of NAACL-HLT 2018*. ACL, New Orleans, LA, 809–819. DOI= https://doi.org/10.18653/v1/N18-1074

[6] Wang, W. Y. 2017. "Liar, liar pants on fire": A new benchmark dataset for fake news detection. In *Proceedings of ACL 2017*. ACL, Vancouver, Canada, 422–426. DOI= https://doi.org/10.18653/v1/P17-2067